

AI Fitbuddy

**LAKSHMI NARAYAN COLLEGE OF TECHNOLOGY &
EXCELLENCE**

Artificial intelligence and machine learning

Submitted To:

SKILL WALLET

Submitted By:

Shivanshu Yadav

Uday Singh Prajapati

Vatan Chavke

Piyush Chouksey

Academic Year : 2026

Section 1 - Abstract (Project Summary)

Project Title: *AI FitBuddy*

FitBuddy is an AI-powered web application designed to generate personalized workout plans and nutrition recommendations based on user fitness goals. The system leverages Google's Gemini AI models to dynamically generate structured fitness routines tailored to individual user preferences such as weight, age, fitness goals, and workout intensity.

The application is developed using FastAPI for backend services, SQLite for database storage, and Jinja2 templates for frontend rendering. FitBuddy allows users to generate customized 7-day workout plans, receive nutrition tips, and update plans based on feedback. The system demonstrates how generative AI can be integrated with modern web frameworks to provide intelligent health and wellness assistance.

SECTION 2 — PROJECT DESCRIPTION

2.1 System Overview

FitBuddy is an AI-powered fitness planning system designed to help users generate customized workout routines based on their personal fitness goals. Many individuals struggle to create structured exercise programs that match their current fitness levels and desired outcomes. FitBuddy addresses this challenge by utilizing generative AI to automatically design effective and personalized workout schedules.

The system allows users to enter personal information such as age, weight, fitness goals (for example weight loss, muscle gain, or general wellness), and preferred workout intensity levels. Using these inputs, the system communicates with Google Gemini AI models to generate a structured seven-day workout plan that includes warm-up exercises, main workout routines, and cooldown activities.

The application also provides complementary nutrition or recovery tips to support the user's fitness goals. In addition, users can submit feedback to modify their workout plan, enabling the AI to generate updated routines based on evolving preferences.

2.2 Problem Domain

Designing an effective fitness plan requires knowledge of exercise science, body conditioning, and nutrition. Beginners often lack this expertise and may follow random workout routines from the internet, which may not be suitable for their specific fitness levels or goals.

Some common problems include:

- Lack of structured workout guidance
- Limited access to professional trainers
- Difficulty maintaining workout consistency
- Lack of personalized fitness recommendations

Fitness trainers and professional programs can be expensive and not easily accessible to everyone. Additionally, many individuals prefer flexible and digital solutions that can provide instant recommendations without requiring appointments with trainers.

Artificial Intelligence offers a promising solution to these challenges. By using generative AI models, it is possible to automatically generate personalized workout routines that adapt to user input and preferences.

FitBuddy aims to bridge this gap by offering an intelligent fitness planning assistant that generates personalized workout plans quickly and efficiently.

2.3 Core Technology Used

FitBuddy system integrates multiple modern technologies to build a full-stack AI-powered web application.

Python

Python is used as the primary programming language due to its simplicity, extensive libraries, and strong support for AI integration.

FastAPI

FastAPI is a modern Python web framework used to build the backend API. It provides high performance, asynchronous capabilities, and automatic API documentation.

Google Gemini AI

Google Gemini AI models are used to generate personalized workout plans and nutrition suggestions based on user inputs.

SQLite Database

SQLite is used as the database system for storing user information and generated workout plans.

SQLAlchemy

SQLAlchemy ORM is used to interact with the SQLite database using Python objects rather than raw SQL queries.

Jinja2 Templates

Jinja2 is used to render dynamic HTML pages that display workout plans and user data.

2.4 Key Capabilities

The FitBuddy system provides several intelligent features designed to enhance user fitness planning.

Key capabilities include:

- Generating personalized workout plans using AI
- Providing nutrition and recovery tips based on fitness goals
- Allowing users to update workout plans through feedback
- Storing user data and generated plans in a database
- Displaying all users and workout plans through an administrative interface

2.5 Target Users

FitBuddy is designed for a wide range of users who want personalized workout guidance.

The primary target users include:

Fitness Beginners

Individuals who are new to exercising and need guidance on how to start a structured workout routine.

Health Enthusiasts

Users who regularly exercise and want AI-generated workout plans tailored to their fitness goals.

Students and Busy Professionals

Individuals who prefer quick and automated fitness planning tools instead of consulting trainers.

Fitness Trainers

Trainers who may use the system as a tool to quickly generate workout plan ideas for clients.

2.6 Real-World Relevance

Artificial intelligence is increasingly being integrated into healthcare and fitness applications. Many modern fitness platforms use AI to recommend workouts, monitor progress, and personalize training routines.

FitBuddy demonstrates a simplified implementation of such systems by combining AI models with web technologies to create an intelligent fitness assistant. The project highlights the potential of generative AI in health-related applications and provides a foundation for future improvements such as wearable integration, mobile apps, and personalized progress tracking.

3. APPLICATION SCENARIOS / USE CASES

3.1 Personal Fitness Assistance

One of the most common use cases of FitBuddy is personal fitness planning for individuals who want structured workout routines but lack professional guidance.

For example, a college student who wants to improve physical fitness may not know how to structure a weekly workout routine. Without proper guidance, the student might perform random exercises without achieving desired results. By using FitBuddy, the student can input personal information such as age, weight, fitness goals, and workout intensity preference.

The system processes this information and generates a personalized seven-day workout plan using the Gemini AI model. The generated plan typically includes:

- Warm-up exercises to prepare muscles
- Main workout activities designed to target specific fitness goals
- Cooldown exercises to prevent injuries and improve recovery

This structured approach ensures that users follow a balanced workout routine that gradually improves their fitness levels.

3.2 Workout Plan Customization Through Feedback

Fitness goals often evolve over time. A user may initially request a workout plan for weight loss but later decide to focus more on cardiovascular endurance or flexibility.

FitBuddy allows users to modify their workout plans by providing feedback such as:

- “Add more cardio exercises”

- “Include yoga sessions”
- “Increase rest days”

Once feedback is submitted, the system uses the Gemini AI model to regenerate an updated workout plan based on the user’s feedback. This dynamic update capability ensures that users receive adaptive workout routines that align with their changing fitness needs.

3.3 Nutrition and Recovery Guidance

Exercise alone is not sufficient to achieve fitness goals. Proper nutrition and recovery practices are equally important.

FitBuddy provides AI-generated nutrition tips based on the user’s selected fitness goal. For example:

- For muscle gain: increase protein intake after workouts
- For weight loss: maintain a calorie deficit and increase hydration
- For general wellness: maintain balanced nutrition with adequate vitamins and minerals

3.4 Administrative Monitoring

In addition to user-facing features, FitBuddy includes an administrative interface that allows administrators to monitor system usage.

The administrator dashboard displays information such as:

- Registered users
- Generated workout plans
- Updated workout plans
- Feedback submissions

This functionality is useful for analyzing user engagement and improving the system over time

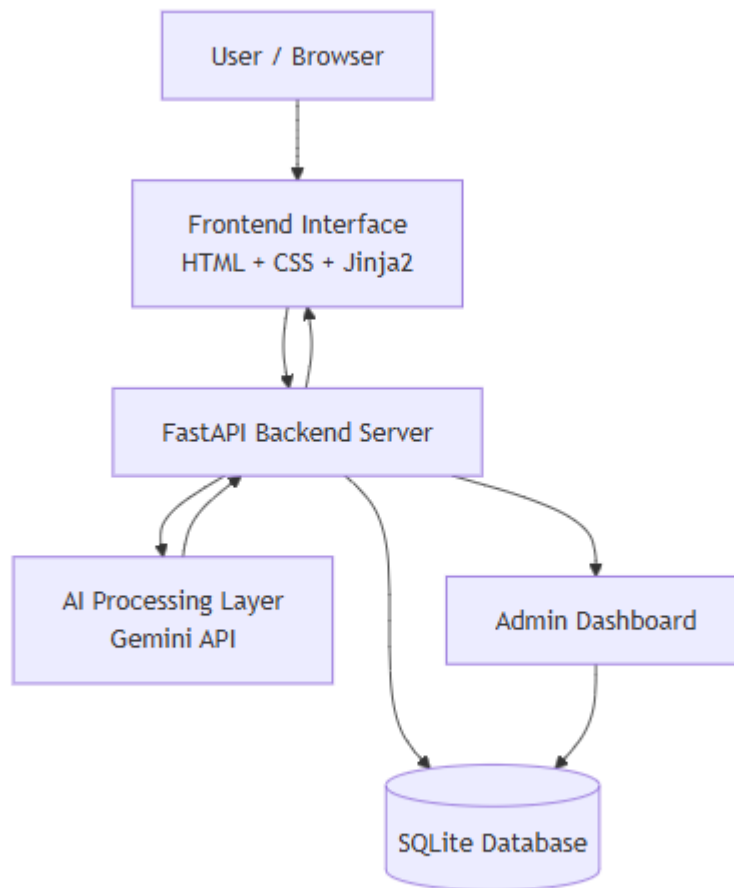
4. TECHNICAL ARCHITECTURE OVERVIEW

4.1 High-Level Architecture

The FitBuddy system consists of four primary components:

1. **Frontend Interface**
2. **Backend API**
3. **Artificial Intelligence Layer**
4. **Database Storage**

The interaction between these components enables seamless generation and storage of personalized workout plans.



4.2 Frontend Layer

The frontend layer is responsible for collecting user input and displaying results.

This layer is built using:

- HTML
- CSS
- Jinja2 templating engine

Users interact with a form where they provide the following details:

- Name
- Age
- Weight
- Fitness goal
- Preferred workout intensity

The frontend sends this information to the backend through HTTP POST requests.

4.3 Backend Layer

The backend of the system is implemented using **FastAPI**, a modern Python web framework known for high performance and automatic API documentation.

The backend performs several critical tasks:

- Receiving user input
- Validating form data
- Sending prompts to the Gemini AI model
- Storing user data in the database
- Rendering results to the user interface

FastAPI handles routing through API endpoints such as:

```
/generate  
/update-plan  
/view-all-users
```

These endpoints coordinate the interaction between the user interface, AI services, and database storage.

4.4 AI Integration Layer

The AI integration layer communicates with Google’s Gemini API to generate workout plans and nutrition tips.

This layer sends prompts to the AI model containing user information such as fitness goals and workout intensity. The AI processes the prompt and generates a structured workout plan that is returned to the backend.

The generated plan typically includes:

- Daily workout routines
- Exercise descriptions
- Suggested repetitions or duration
- Warm-up and cooldown instructions

.

4.5 Database Layer

FitBuddy uses **SQLite**, a lightweight relational database system that is suitable for small to medium-scale applications.

The database schema consists of two main tables:

User Table

Stores basic user information.

Id	Unique user identifier
Name	User name
Age	User age
Weight	User weight

Goal	Fitness goal
Intensity	Preferred workout intensity

WorkoutPlan Table

Stores generated workout plans.

Id	Plan Identifier
User_id	Associated user
Original_plan	Initial generated workout plan
Updated_plan	Plan updated through feedback

5. PREREQUISITES

5.1 Software Requirements

The following software tools are required to develop and run the application.

Python

Python is the primary programming language used for developing the application.

Recommended version:

```
Python 3.10 or higher
```

Integrated Development Environment (IDE)

A suitable IDE is required for writing and debugging code.

Recommended IDE:

- Visual Studio Code

FastAPI Framework

FastAPI is used to build the backend server and manage API routes.

Key advantages include:

- High performance
- Asynchronous request handling
- Automatic API documentation

Uvicorn Server

Uvicorn is an ASGI server used to run FastAPI applications.

The server is started using the command:

```
uvicorn app.main:app --reload
```


5.2 Libraries / Frameworks / Dependencies

The FitBuddy application relies on several Python libraries.

FastAPI

Provides the core backend framework for building the web API.

SQLAlchemy

Handles database operations using an Object Relational Mapping (ORM) system.

Jinja2

Allows dynamic HTML rendering with variables and loops.

Python-Multipart

Handles form submissions from HTML forms.

Google GenAI

Provides integration with the Gemini AI models.

Python-Dotenv

Allows environment variables such as API keys to be securely stored in .env files.

Example dependency installation command:

pip install fastapi uvicorn sqlalchemy jinja2 python-multipart google-genai python-dotenv

5.3 Hardware Requirements

FitBuddy does not require specialized hardware.

Minimum recommended system specifications include:

- Processor: Dual-core CPU
- RAM: 4 GB
- Storage: 2 GB free space
- Internet connection for AI API communication

6. PRIOR KNOWLEDGE REQUIRED

6.1 Python Programming

Python is the primary programming language used in the development of the FitBuddy system. A basic understanding of Python programming concepts is necessary to implement and maintain the application.

Developers working on this project should be familiar with fundamental Python concepts such as:

- Variables and data types
- Functions and modules
- Object-Oriented Programming (OOP) concepts
- File handling
- Exception handling

Python's simplicity and extensive ecosystem make it a popular choice for building web applications and integrating artificial intelligence services.

In the FitBuddy project, Python is used to:

- Implement backend logic using FastAPI
- Connect with the Gemini AI API
- Manage database interactions using SQLAlchemy
- Process user input and generate responses

6.2 Web Development Concepts

Since FitBuddy is a web-based application, developers should understand basic web development concepts.

Important concepts include:

- HTTP request and response mechanisms
- Client-server architecture
- Form submission using POST requests
- Rendering dynamic web pages

The frontend of FitBuddy uses **HTML and CSS** to create the user interface. These technologies help design interactive forms that allow users to input fitness information and view generated workout plans.

6.3 FastAPI Framework

FastAPI is the primary web framework used for backend development in this project. Understanding FastAPI is important for implementing API routes and handling web requests.

Developers should understand:

- API routing
- Request handling
- Response generation
- Dependency injection
- Middleware concepts

FastAPI also provides automatic API documentation through Swagger UI, which allows developers to test API endpoints directly from the browser.

6.4 Database Management

FitBuddy uses SQLite for storing user information and generated workout plans. Basic knowledge of database systems is required to understand how data is stored and retrieved.

Developers should understand:

- Relational database concepts
- Tables and relationships
- SQL queries
- CRUD operations (Create, Read, Update, Delete)

In this project, **SQLAlchemy ORM** is used to interact with the SQLite database. ORM allows developers to manipulate database records using Python objects instead of writing raw SQL queries.

6.5 Artificial Intelligence APIs

The FitBuddy system integrates **Google Gemini AI models** to generate workout plans and nutrition suggestions.

Developers should understand the following concepts related to AI APIs:

- API keys and authentication
- Prompt engineering
- Request and response handling
- AI-generated content processing

By sending prompts containing user fitness information to the Gemini API, the system receives structured responses that are formatted and displayed to users.

6.6 Version Control Systems

Version control systems help developers manage and track changes in the project codebase. FitBuddy development uses **Git and GitHub** for version control.

Important version control concepts include:

- Git repositories

- Commits and branches
- Pull and push operations
- Collaboration through GitHub

Using Git ensures that the project code is properly managed and allows multiple developers to collaborate effectively.

7. PROJECT OBJECTIVES

7.1 Technical Objectives

The primary technical objective of the FitBuddy project is to design and implement an AI-powered web application that can generate personalized workout plans using generative artificial intelligence models.

Key technical objectives include:

- Developing a web-based fitness planning system using FastAPI
 - Integrating Google Gemini AI models for workout plan generation
 - Implementing database storage for user information and generated plans
 - Designing a clean and user-friendly interface for interacting with the system
-

7.2 Functional Objectives

The functional objectives define what the system should be capable of performing.

These include:

- Accepting user fitness information through a web interface
 - Generating personalized workout plans using AI
 - Providing goal-based nutrition and recovery tips
 - Allowing users to update workout plans through feedback
 - Storing and managing user data in a database
-

7.3 Learning Objectives

The project also aims to provide valuable learning outcomes for developers and students working on the system.

These learning outcomes include:

- Understanding how AI models can be integrated into web applications
 - Learning modern Python backend development using FastAPI
 - Working with relational databases using SQLAlchemy
 - Implementing real-world AI-powered software solutions
-

7.4 Performance Objectives

Another objective of the system is to ensure efficient performance when generating workout plans and processing user requests.

Performance goals include:

- Fast response time for AI-generated plans
- Efficient database queries
- Smooth user interaction through the web interface

8. SYSTEM WORKFLOW

8.1 Overview of System Workflow

The FitBuddy system follows a structured workflow that enables users to generate personalized fitness plans using artificial intelligence. The workflow begins when a user accesses the application through a web browser and enters their fitness details into the system.

Once the user submits the form, the backend server processes the request, communicates with the Gemini AI model to generate a workout plan, stores the generated information in a database, and then displays the results to the user.

This process ensures that users receive customized fitness guidance based on their goals, experience level, and workout preferences.

The workflow is designed to ensure:

- Efficient processing of user requests
 - Accurate AI-generated responses
 - Reliable storage of generated plans
 - Smooth interaction between frontend and backend components
-

8.2 Step-by-Step Workflow

The complete workflow of the FitBuddy system consists of several steps.

Step 1 – User Accesses the Application

The workflow begins when a user opens the FitBuddy application in a web browser.

The homepage provides a form where users can input their fitness information, including:

- Name
- Age
- Weight
- Fitness goal (e.g., weight loss or muscle gain)
- Workout intensity preference (low, medium, or high)

The form is designed to collect essential information required for generating a personalized workout plan.

Step 2 – User Submits Fitness Information

After entering the required details, the user submits the form.

The frontend sends the collected data to the FastAPI backend using an HTTP POST request.

Example endpoint used:

`/generate`

This request contains the user's information, which will be used to generate a customized fitness plan.

Step 3 – Backend Receives and Processes Request

The FastAPI backend receives the request and validates the input data to ensure that all required fields are present.

The backend then prepares a structured prompt containing the user's fitness details. This prompt is designed to instruct the AI model to generate a suitable workout plan.

Example prompt structure:

Generate a 7-day workout plan for a 22-year-old user weighing 75 kg who wants to lose weight with medium intensity workouts.

Step 4 – Gemini AI Generates Workout Plan

The Gemini AI model analyzes the prompt and generates a detailed workout plan.

The generated plan typically includes:

- Warm-up exercises
- Main workout routines
- Sets and repetitions
- Cooldown activities
- Rest day recommendations

The AI ensures that the generated plan aligns with the user's selected fitness goal and intensity level.

Once the response is generated, the AI sends the structured workout plan back to the backend server.

Step 5 – Backend Generates Nutrition Tip

In addition to the workout plan, the system also generates a nutrition or recovery tip based on the user's fitness goal.

For example:

- Weight loss: increase water intake and reduce processed foods
- Muscle gain: consume protein-rich foods after workouts
- General wellness: maintain balanced nutrition and hydration

These tips help users improve their overall fitness results by aligning diet and recovery practices with their training routine.

Step 6 – Data Storage in Database

Once the workout plan and nutrition tip are generated, the backend stores the user's information and the generated plan in the SQLite database.

The database ensures that all user interactions are stored for future reference.

Stored data includes:

- User details
- Original workout plan
- Updated workout plan (if modified through feedback)
- Nutrition tip

This allows the system to maintain records of generated plans and track user interactions.

Step 7 – Result Rendering

After storing the generated information, the backend sends the workout plan and nutrition tip to the frontend.

The frontend uses **Jinja2 templates** to dynamically render the results on the result page.

The result page displays:

- The personalized workout plan
- Nutrition recommendations
- Options to update the workout plan through feedback

This provides users with an easy-to-read structured fitness plan.

Step 8 – Feedback-Based Plan Update (Optional)

If the user wants to modify the workout plan, they can submit feedback through the application.

Examples of feedback include:

- “Add more cardio”
- “Include yoga sessions”
- “Increase rest days”

The backend sends the original workout plan along with the user’s feedback to the Gemini AI model.

The AI then generates an updated version of the workout plan that incorporates the requested modifications.

The updated plan is stored in the database and displayed to the user.

Step 9 – Administrative Monitoring

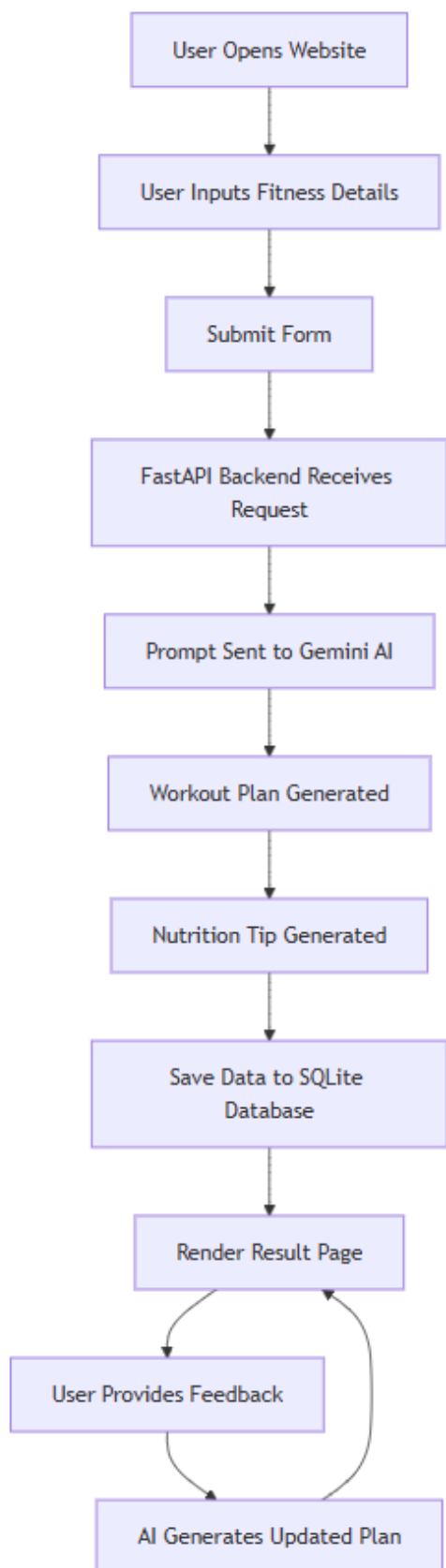
The system also provides an administrative view where administrators can see all users and their associated workout plans.

The admin interface displays information such as:

- User details
- Original workout plan
- Updated workout plan

This feature allows administrators to monitor system usage and analyze user engagement.

8.3 System Workflow Diagram



8.4 Advantages of the Workflow Design

The workflow design of the FitBuddy system offers several advantages.

Automation

AI automatically generates workout plans without requiring manual intervention.

Personalization

Workout routines are tailored to individual user preferences.

Scalability

The system architecture allows integration with larger databases or cloud deployment.

Flexibility

Users can modify their plans through feedback, enabling continuous improvement of workout routines.

9. MILESTONE 1 – REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition

In modern society, maintaining physical fitness has become increasingly important due to sedentary lifestyles and rising health concerns. Many individuals wish to improve their health by exercising regularly but often face difficulties in creating structured workout plans that align with their fitness goals.

Designing an effective workout routine requires knowledge of exercise science, body conditioning, and proper recovery practices. Beginners often lack this expertise and may follow random workout routines found online, which may not suit their specific fitness levels or health conditions.

Several common challenges faced by individuals include:

- Lack of knowledge about structured workout planning
- Limited access to professional fitness trainers
- High cost of personal training programs
- Difficulty maintaining consistent workout routines
- Lack of personalized fitness guidance

Traditional fitness applications often provide pre-defined workout routines that do not adapt to individual needs. These static programs may not be suitable for users with different fitness levels, body conditions, or personal preferences.

Artificial intelligence provides an opportunity to solve this problem by generating personalized workout plans dynamically based on user inputs. By leveraging generative AI models such as Google Gemini, it is possible to create customized fitness plans that adapt to the user's goals, preferences, and feedback.

The FitBuddy system is designed to address these challenges by providing an AI-powered platform that generates personalized workout plans and nutrition tips for users. The system aims to make fitness planning more accessible, efficient, and adaptive.

9.2 Functional Requirements

Functional requirements describe the specific features and functionalities that the system must provide.

The FitBuddy system includes the following core functional requirements.

User Input Collection

The system must allow users to enter their personal fitness details through a web interface. The required inputs include:

- Name
- Age
- Weight
- Fitness goal
- Workout intensity preference

These inputs are essential for generating personalized workout plans.

Workout Plan Generation

The system must generate a structured workout plan based on the user's fitness details. This plan should include:

- Warm-up exercises
- Main workout routines
- Exercise repetitions or durations

- Cooldown activities
- Rest day recommendations

The workout plan should cover a seven-day schedule tailored to the user's fitness goal.

Nutrition Tip Generation

The system must provide nutrition or recovery tips aligned with the user's selected fitness goal. These tips help users maintain proper diet and recovery practices.

Examples include:

- Protein intake recommendations for muscle gain
 - Hydration suggestions for endurance training
 - Balanced diet advice for overall wellness
-

Feedback-Based Plan Update

The system must allow users to submit feedback regarding their workout plan.

Based on the feedback, the system should regenerate an updated workout plan that incorporates the user's suggestions.

Database Storage

The system must store user information and generated workout plans in a database.

This enables the system to:

- Maintain records of generated plans
- Track user interactions

- Display data in the administrative interface
-

Administrative Dashboard

The system must provide an administrative view where administrators can see a list of users and their generated workout plans.

This allows administrators to monitor the usage of the system and analyze user engagement.

9.3 Non-Functional Requirements

Non-functional requirements describe the quality attributes and constraints of the system.

Performance

The system should generate workout plans quickly after receiving user input. All responses should be delivered within a few seconds to ensure smooth user interaction.

Reliability

The system must consistently generate accurate and relevant workout plans based on user input.

Error handling mechanisms should be implemented to handle invalid inputs or API failures.

Usability

The user interface should be simple and intuitive, allowing users to easily enter their fitness details and view generated workout plans.

Clear instructions and structured outputs should improve user experience.

Security

Sensitive data such as API keys should not be exposed in the application code. Instead, environment variables should be used to securely store credentials.

Scalability

The system architecture should allow future expansion, such as integrating cloud databases, supporting mobile applications, or adding advanced fitness tracking features.

9.4 Use Case Analysis

Use case analysis describes how different users interact with the system.

The primary actor in the FitBuddy system is the **User**, while a secondary actor is the **Administrator**.

Use Case 1 – Generate Workout Plan

Actor: User

Description:

The user enters personal fitness details and receives a personalized workout plan generated by the AI model.

Steps:

1. User opens the FitBuddy application.
2. User enters fitness details.
3. User submits the form.
4. Backend sends prompt to Gemini AI.

5. AI generates workout plan.
 6. System displays workout plan to the user.
-

Use Case 2 – Update Workout Plan

Actor: User

Description:

The user provides feedback on the generated workout plan, and the AI regenerates an updated version.

Steps:

1. User reviews the generated workout plan.
 2. User submits feedback.
 3. Backend sends original plan and feedback to AI.
 4. AI generates an updated plan.
 5. System displays updated plan.
-

Use Case 3 – View All Users

Actor: Administrator

Description:

The administrator views all users and their workout plans through the admin dashboard.

9.5 System Design Decisions

Several design decisions were made during the development of FitBuddy to ensure efficiency, scalability, and simplicity.

Selection of FastAPI

FastAPI was chosen as the backend framework because of its high performance, ease of use, and support for asynchronous programming.

It also provides automatic API documentation through Swagger UI, which simplifies testing and debugging.

Selection of SQLite

SQLite was selected as the database system due to its lightweight nature and ease of setup. It is suitable for small to medium-scale applications and does not require a separate database server.

Selection of Gemini AI

Google Gemini AI was chosen as the generative model for workout plan generation because of its advanced natural language processing capabilities.

The model can generate detailed workout plans and nutrition tips based on user prompts.

Modular Project Structure

The project was structured into separate modules to improve maintainability and scalability.

Key modules include:

- API routes
- Database operations
- AI generation logic
- HTML templates

This modular design ensures that different components of the system can be modified independently without affecting other parts of the application.

9.6 Technology Stack Justification

Python	Core programming language
FastAPI	Backend web Framework
Gemini AI	Workout plan generation
SQLite	Data Storage
SQLAlchemy	Database ORM
Jinja2	Dynaic HTML rendering
Uvicorn	Application server

Sequence Diagram:

10. MILESTONE 2 – ENVIRONMENT SETUP & INITIAL CONFIGURATION

10.1 Development Environment Setup

Before developing the FitBuddy application, the necessary software tools and development environment were configured.

The development process was carried out using the following tools:

- **Python Programming Language**
- **Visual Studio Code (VS Code)**
- **Virtual Environment (venv)**
- **Git and GitHub for version control**

These tools provide an efficient environment for writing, testing, and managing application code.

Python Installation

Python is the core programming language used for the FitBuddy system. Python provides extensive support for web development, database integration, and artificial intelligence APIs.

To verify that Python is installed correctly, the following command can be executed in the terminal:

```
python --version
```

```
Python 3.10 or higher
```

Integrated Development Environment (IDE)

Visual Studio Code (VS Code) was used as the primary development environment for this project.

VS Code offers several advantages for Python development, including:

- Syntax highlighting
- Integrated debugging tools
- Extension support for Python and FastAPI
- Built-in Git integration
- Terminal access within the editor

These features make VS Code a suitable environment for developing modern web applications.

10.2 Creating a Virtual Environment

A virtual environment is used to isolate project dependencies from the global Python installation. This ensures that libraries used by the FitBuddy project do not conflict with other Python projects on the system.

To create a virtual environment, the following command was executed:

```
python -m venv fitbuddy-env
```

After creating the virtual environment, it must be activated before installing project dependencies.

Activating Virtual Environment (Windows)

```
fitbuddy-env\Scripts\activate
```

10.3 Installing Required Libraries

After activating the virtual environment, the required Python libraries were installed using the **pip package manager**.

The following command installs all necessary dependencies for the FitBuddy application:

```
pip install fastapi uvicorn jinja2 sqlalchemy python-multipart python-dotenv google-genai
```

Library	Purpose
FastAPI	Backendweb framework
Uvicorn	ASGI server to run the application
Jinja2	Template engine for rendering HTML
SQLaichemy	Object relational Mapper for database operations
Python-multipart	Handles HTML form data
Python-dotenv	Loads environment Variables
Google-genai	Integration with Google Gemini AI models

10.4 Project Directory Structure

After installing the dependencies, the project directory was organized into a structured format. Proper project organization helps maintain code readability and scalability.

The basic project structure for FitBuddy is shown below:

```
fitbuddy/
├── app/
│   ├── main.py
│   ├── routes.py
│   ├── database.py
│   ├── gemini_generator.py
│   ├── gemini_flash_generator.py
│   └── updated_plan.py
├── templates/
│   ├── index.html
│   ├── result.html
│   └── all_users.html
├── static/
│   └── images/
├── .env
├── requirements.txt
└── README.md
```

Explanation of Project Components

app/

This directory contains the main backend logic of the application.

Important files include:

- **main.py** – entry point of the FastAPI application
- **routes.py** – handles API endpoints and request processing
- **database.py** – manages database connections and models
- **gemini_generator.py** – generates workout plans using Gemini AI
- **gemini_flash_generator.py** – generates nutrition tips
- **updated_plan.py** – updates workout plans based on feedback

templates/

The templates directory contains HTML files used to render the user interface.

These templates are processed using the **Jinja2 template engine** to dynamically display data generated by the backend.

Important templates include:

- **index.html** – main user input form
- **result.html** – displays generated workout plans
- **all_users.html** – administrative view of users and plans

static/

The static directory contains static resources such as images and styling assets used by the frontend.

.env

The .env file stores environment variables such as API keys and configuration settings.

```
GEMINI_API_KEY=your_api_key_here
```

Storing API keys in environment variables improves security by preventing sensitive information from being exposed in source code.

10.5 Running the FastAPI Server

Once the environment is configured and dependencies are installed, the FastAPI application can be started using the Uvicorn server.

The following command is used to start the server:

```
uvicorn app.main:app --reload
```

The `--reload` option enables automatic server reloading whenever code changes are detected during development.

Once the server starts successfully, the application can be accessed at:

```
http://127.0.0.1:8000
```

10.6 API Documentation

One of the advantages of FastAPI is that it automatically generates interactive API documentation.

Developers can access this documentation through:

<http://127.0.0.1:8000/docs>

The documentation interface allows developers to:

- View available API endpoints
- Test API requests
- Inspect request parameters
- Examine API responses

This feature simplifies debugging and development of backend services.

10.7 Dependency Management

To ensure reproducibility of the project environment, the installed dependencies were exported into a **requirements file**.

The following command was used:

```
pip freeze > requirements.txt
```

The requirements.txt file allows other developers to recreate the project environment by running:

```
pip install -r requirements.txt
```

11. MILESTONE 3 – CORE SYSTEM DEVELOPMENT

11.1 Backend Implementation using FastAPI

The backend of the FitBuddy system is implemented using **FastAPI**, a modern Python web framework designed for building high-performance APIs.

FastAPI was selected because it offers several advantages, including:

- High performance comparable to Node.js and Go
- Easy integration with Python libraries
- Automatic API documentation
- Built-in data validation
- Support for asynchronous operations

The backend server is responsible for handling user requests, interacting with the Gemini AI model, managing database operations, and rendering results through HTML templates.

The entry point of the application is defined in the **main.py** file.

Example structure of the main application file:

```
from fastapi import FastAPI
from app.routes import router

app = FastAPI()

app.include_router(router)
```

11.2 API Routing and Request Handling

The FitBuddy application uses API routes to process user requests.

Routes are defined in the **routes.py** file, which contains functions responsible for handling different application endpoints.

Important endpoints include:

Endpoint	Description
/	Displays the homepage
/generate	Generates workout plan
/update-plan	Updates workout plan using feedback
/view-all-users	Displays all stored users

These routes allow users and administrators to interact with the system through a web interface.

Example route for generating a workout plan:

```
@router.post("/generate")
def generate_plan(
    request: Request,
    name: str = Form(...),
    age: int = Form(...),
    weight: int = Form(...),
    goal: str = Form(...),
    intensity: str = Form(...)
):
```

11.3 Integration with Gemini AI

One of the most important components of the FitBuddy system is the integration with **Google Gemini AI models**.

The AI model is responsible for generating personalized workout plans and nutrition tips based on user input.

The integration is implemented in the file:

gemini_generator.py

The generator function constructs a prompt containing the user's fitness details and sends it to the Gemini API.

Example prompt:

Create a detailed 7-day workout plan for a user with the following details:

Age: 25

Weight: 70kg

Goal: Weight Loss

Workout Intensity: Medium

The AI processes this prompt and returns a structured workout plan containing exercise recommendations.

Example AI request implementation:

```
response = client.models.generate_content(  
    model="gemini-1.5-flash",  
    contents=prompt  
)
```

The response received from the AI model is then formatted and returned to the frontend for display.

11.4 Workout Plan Generator

The workout plan generator is responsible for creating personalized weekly exercise routines based on user input.

The system uses the following parameters to generate the plan:

- Age
- Weight
- Fitness goal
- Workout intensity

The generated workout plan typically includes:

- Warm-up exercises
- Strength training exercises
- Cardiovascular workouts
- Core workouts
- Rest and recovery days

Example output format:

Day 1: Full Body Workout

Warm-up: 10 minutes jogging

Exercises: Push-ups, Squats, Lunges

Cooldown: Stretching

Day 2: Cardio

30 minutes running

Core exercises

This structured output ensures that users receive a clear and organized fitness schedule.

11.5 Nutrition Tip Generator

In addition to workout plans, FitBuddy provides AI-generated nutrition tips to support the user's fitness goals.

The nutrition tip generator uses a separate AI prompt designed to produce concise health advice.

Example prompt:

Provide a short nutrition tip for someone focusing on muscle gain.

Example response:

Include high-protein foods such as eggs, chicken, and legumes in your post-workout meals to support muscle recovery and growth.

These tips help users maintain proper dietary habits that complement their training routines.

11.6 Feedback-Based Plan Updating

The system also allows users to modify their workout plans by submitting feedback.

Examples of feedback include:

- "Add more cardio exercises"
- "Include yoga sessions"
- "Reduce workout intensity"

The system sends both the original workout plan and the user's feedback to the AI model.

Example prompt for updating plans:

Modify the following workout plan based on this feedback:

Feedback: Add more cardio exercises.

The AI generates a revised workout plan that incorporates the requested changes.

This feature allows the system to provide dynamic and adaptive workout recommendations.

11.7 Database Implementation

To store user data and generated workout plans, the FitBuddy system uses **SQLite** as its database.

Database operations are managed using **SQLAlchemy**, which provides an Object Relational Mapping (ORM) system.

The database models are defined in the **database.py** file.

Example user model:

11.8 Database Operations

Several database operations are implemented to manage user data and workout plans.

Key operations include:

Saving User Information

When a user submits their fitness details, the system creates a new user record in the database.

Saving Workout Plans

After generating the workout plan, the system stores the plan in the database along with the associated user ID.

Updating Workout Plans

When feedback is submitted, the updated workout plan is stored in the database while preserving the original plan.

These operations ensure that all user interactions with the system are recorded.

11.9 Frontend Integration

The frontend interface of the FitBuddy system is built using **HTML and Jinja2 templates**.

Jinja2 allows dynamic data rendering by embedding Python variables inside HTML templates.

Example template syntax:

```
<h2>Your Workout Plan</h2>
<p>{{ plan }}</p>
```

Template	Purpose
index.html	User input form
result.html	Displays generated workout plan
all_users.html	Shows stored users and plans

11.10 User Interface Design

The user interface of the FitBuddy application was designed to be simple and user-friendly.

Key design principles include:

- Minimalistic layout
- Easy-to-understand form inputs
- Clear display of generated workout plans
- Responsive design for different screen sizes

12. MILESTONE 4 – INTEGRATION & OPTIMIZATION

After completing the development of individual components, the next step in the FitBuddy project involved integrating all modules into a single functioning system. Integration ensures that the frontend interface, backend API, AI services, and database interact smoothly with one another.

In addition to integration, several optimizations were implemented to improve performance, security, and reliability of the system.

12.1 Component Integration

The FitBuddy system consists of multiple independent components that must work together to deliver the desired functionality. These components include:

- User Interface (Frontend)
- Backend API (FastAPI)
- AI Integration Layer (Gemini API)
- Database Storage (SQLite)

Each component performs a specific role, and integration ensures proper communication between them.

Frontend and Backend Integration

The frontend interface collects user input through HTML forms. When a user submits the form, the data is sent to the backend using HTTP POST requests.

The backend receives this data and processes it using FastAPI routes. The backend then communicates with the AI module to generate workout plans.

Once the response is received, the backend renders the results using Jinja2 templates and sends them back to the frontend.

Backend and AI Integration

The backend communicates with the Gemini AI API to generate workout plans and nutrition tips.

This integration involves sending a prompt containing user fitness details to the AI model and receiving a generated response.

The backend processes the AI response and formats it before displaying it to the user.

Backend and Database Integration

The backend interacts with the SQLite database using SQLAlchemy ORM.

Database integration allows the system to:

- Store user details
- Save generated workout plans
- Update plans based on feedback
- Retrieve stored user data for the admin dashboard

This integration ensures that user data persists across sessions.

12.2 System Integration Workflow

The integrated system follows a structured workflow where each component interacts sequentially.

The typical workflow includes the following steps:

1. User enters fitness details through the web interface.
2. Frontend sends data to the FastAPI backend.
3. Backend sends prompt to Gemini AI.
4. AI generates workout plan and returns response.
5. Backend stores the data in SQLite database.
6. Backend renders the results using Jinja2 templates.
7. User views the generated workout plan and nutrition tips.

This integrated workflow ensures seamless operation of the application.

12.3 Performance Optimization

To ensure smooth performance of the system, several optimization strategies were implemented during development.

Efficient API Calls

The AI prompt structure was designed carefully to minimize unnecessary processing while still generating detailed workout plans. Short and structured prompts reduce response time from the AI model.

Lightweight Database System

SQLite was chosen because it is lightweight and does not require a separate database server. This improves performance for small to medium-scale applications like FitBuddy.

Modular Code Structure

The project was divided into separate modules such as:

- routes
- AI generators
- database operations
- HTML templates

This modular architecture reduces code complexity and improves maintainability.

2.5 Security Considerations

Environment Variables for API Keys

The Gemini API key is stored in an environment file (.env) rather than hardcoded into the application.

Example:

GEMINI_API_KEY=your_api_key_here

This prevents sensitive information from being exposed in the source code.

Input Validation

User input is validated before processing to prevent invalid data from affecting system behavior.

This ensures that the backend receives only properly formatted information.

Limited Data Exposure

The system only stores necessary information such as user fitness details and generated workout plans. Sensitive personal data is not collected.

[12.6 System Reliability Improvements](#)

Several steps were taken to improve system reliability.

Automatic Server Reload

During development, the Uvicorn server was run using the reload option:

```
uvicorn app.main:app --reload
```

This automatically restarts the server when code changes are detected, improving development efficiency.

Logging and Debugging

Logs were used during development to identify errors and monitor system behavior.

13. MILESTONE 5 – TESTING & VALIDATION

13.1 Types of Testing Performed

Several types of testing were performed during the development of the FitBuddy system.

13.2 Unit Testing

In the FitBuddy application, several modules were tested individually.

Modules Tested

- 1. Workout Plan Generator**
- 2. Nutrition Tip Generator**
- 3. Database Operations**
- 4. API Routes**

For example, the workout generator module was tested by providing sample inputs and verifying whether the Gemini AI model generated appropriate workout plans.

Example input:

Goal: Weight Loss

Age: 24

Weight: 75 kg

Intensity: Medium

Expected output:

A structured workout plan including cardio exercises, strength training, and rest days.

Unit testing ensured that each function produced expected outputs when given valid inputs.

13.3 Integration Testing

In FitBuddy, integration testing focused on verifying the communication between:

- Frontend interface
- FastAPI backend
- Gemini AI integration
- SQLite database

13.4 System Testing

System testing evaluates the complete application as a whole.

This testing ensures that the entire system behaves as expected when used in a real environment.

During system testing, the following aspects were evaluated:

- Form submission functionality
- AI workout plan generation
- Nutrition tip generation
- Feedback-based plan updates
- Database storage and retrieval
- Admin dashboard functionality

System testing confirmed that the FitBuddy application performs correctly when accessed through a web browser.

13.5 Test Case Table

The following table summarizes the test cases performed during system validation.

Test Case ID	Test Description	Input	Expected Output	Result
TC01	Generate workout plan	Valid user details	Workout plan generated	Pass
TC02	Generate nutrition tip	Goal: muscle gain	Nutrition advice displayed	Pass
TC03	Update workout plan	Feedback: add cardio	Updated plan generated	Pass
TC04	Store user data	Form submission	Data saved in database	Pass
TC05	Admin view users	Access admin route	List of users displayed	Pass
TC06	Invalid input handling	Missing fields	Error message shown	Pass

13.6 Error Handling Validation

During testing, several potential errors were evaluated to ensure the system handles them properly.

Examples include:

Missing Input Fields

If a user submits the form without entering required details, the system returns a validation error.

API Connection Errors

If the AI service fails to respond, the backend handles the error and displays an appropriate message instead of crashing the application.

Database Errors

Database connection failures or query issues are logged and handled to prevent application crashes.

13.7 Performance Testing

Performance testing was conducted to measure how quickly the system generates workout plans and processes user requests.

The results showed that:

- Form submission is processed instantly
- AI-generated workout plans are returned within a few seconds
- Database storage operations complete quickly

These results indicate that the system performs efficiently for normal usage scenarios.

13.8 Testing Tools Used

Several tools were used during the testing phase.

FastAPI Swagger UI

FastAPI automatically generates interactive API documentation that allows developers to test API endpoints directly.

Access URL: <http://127.0.0.1:8000/docs>

This interface allows developers to send requests and verify API responses.

Browser Testing

The application was tested using modern web browsers to verify that the user interface renders correctly.

Debugging Tools

The Visual Studio Code debugger and terminal logs were used to identify and fix errors during development.

13.9 Validation Results

The system successfully performs the following tasks:

- Accepts user fitness information
- Generates personalized workout plans using AI
- Provides nutrition tips based on fitness goals
- Stores user data in the database
- Updates workout plans through feedback
- Displays user information through the admin dashboard

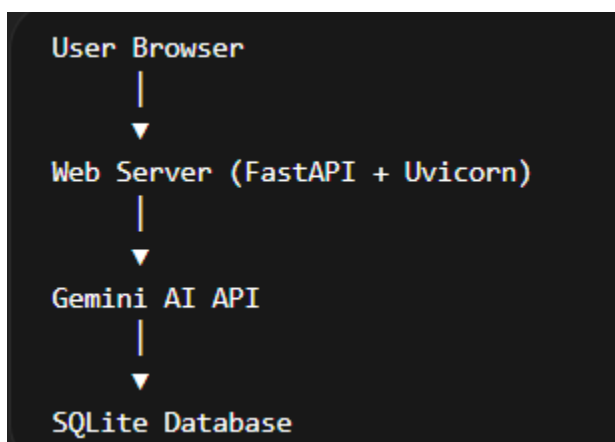
14. DEPLOYMENT

14.1 Deployment Architecture

The deployment architecture of the FitBuddy system follows a client–server model.

In this architecture, the user interacts with the system through a web browser, which communicates with the backend server that processes requests and returns responses.

Deployment Architecture Diagram



14.2 Local Deployment

During development and testing, the FitBuddy system was deployed locally on the developer's machine.

The FastAPI application was executed using the **Uvicorn server** with the following command:

```
uvicorn app.main:app --reload
```

The application could then be accessed through a web browser using the following address:

```
http://127.0.0.1:8000
```

Local deployment allowed developers to test the application, debug issues, and verify system functionality before considering cloud deployment.

14.3 Environment Configuration

Before deployment, environment variables must be configured properly to store sensitive information such as API keys.

Example .env

```
configuration: GEMINI_API_KEY=your_api_key_here
```

Environment variables ensure that sensitive credentials remain secure and are not exposed in the application code.

14.4 Deployment Considerations

Several factors must be considered when deploying the FitBuddy system to production environments.

Scalability

The system architecture should support multiple users simultaneously. Future improvements may involve migrating from SQLite to cloud-based databases such as PostgreSQL.

Security

Sensitive data such as API keys should be stored securely using environment variables.

Secure communication using HTTPS should also be implemented when deploying the system publicly.

Monitoring and Maintenance

Production systems require monitoring to detect potential errors or performance issues. Logging mechanisms and monitoring tools can be used to track system behavior and improve reliability.

15. PROJECT STRUCTURE

Project Folder Structure

```
fitbuddy/
├── app/
│   ├── main.py
│   ├── routes.py
│   ├── database.py
│   ├── gemini_generator.py
│   ├── gemini_flash_generator.py
│   └── updated_plan.py
├── templates/
│   ├── index.html
│   ├── result.html
│   └── all_users.html
├── static/
│   └── images/
├── .env
├── requirements.txt
└── README.md
```

Explanation of Project Files

main.py

This file serves as the entry point of the FastAPI application. It initializes the server and registers API routes.

routes.py

This file defines the API endpoints responsible for handling user requests such as generating workout plans and updating plans based on feedback.

database.py

This module manages database connections and defines database models for storing user information and workout plans.

gemini_generator.py

This file contains the logic for generating workout plans using the Gemini AI model.

gemini_flash_generator.py

This module generates nutrition tips based on user fitness goals.

updated_plan.py

This module processes user feedback and generates updated workout plans.

16. RESULTS

Generated Workout Plan Example

When a user enters fitness information such as age, weight, goal, and workout intensity, the system generates a structured workout plan similar to the following:

Day 1 – Full Body Workout

Warm-up: 10 minutes jogging

Exercises: Push-ups, Squats, Lunges

Cooldown: Stretching

Day 2 – Cardio

30 minutes running

Core exercises

Day 3 – Strength Training

Bench press, Deadlifts, Planks

Nutrition Tip Example

For users aiming for muscle gain, the system may generate a tip such as:

Include high-protein foods such as eggs, chicken, and legumes in your meals to support muscle recovery and growth.

Admin Dashboard Results

The admin interface allows administrators to view all users and their associated workout plans stored in the database.

This functionality helps monitor system usage and manage user data effectively.

17. ADVANTAGES AND LIMITATIONS

Advantages

The FitBuddy system offers several advantages.

- Personalized workout plan generation using AI
 - User-friendly web interface
 - Dynamic plan updates based on feedback
 - Lightweight and efficient backend architecture
 - Database storage for user information
-

Limitations

Despite its advantages, the system has certain limitations.

- Requires internet access for AI API communication
- Limited customization options for advanced athletes
- SQLite database may not scale for large user bases

Future improvements can address these limitations by integrating more advanced features.

18. FUTURE ENHANCEMENTS

The FitBuddy system can be further enhanced by implementing additional features.

Possible future improvements include:

Mobile Application

Developing a mobile version of FitBuddy to allow users to access the system from smartphones.

Wearable Device Integration

Integration with fitness trackers such as smartwatches to monitor user activity and adjust workout plans automatically.

Progress Tracking

Adding features to track user fitness progress over time.

Cloud Database Integration

Replacing SQLite with scalable cloud databases such as PostgreSQL or MongoDB.

AI Personal Trainer Chatbot

Implementing an AI chatbot that provides real-time fitness advice and workout assistance.

19. CONCLUSION

The FitBuddy project demonstrates the integration of modern web technologies with artificial intelligence to create an intelligent fitness planning system.

The system successfully generates personalized workout plans and nutrition recommendations based on user input. By combining FastAPI, SQLite, and Google Gemini AI models, the application provides an efficient and scalable platform for automated fitness guidance.

The project highlights the potential of generative AI in health and wellness applications and provides a foundation for future development of intelligent fitness solutions.

20. REFERENCES

The following resources were used during the development and documentation of the FitBuddy project.

1. FastAPI Documentation
<https://fastapi.tiangolo.com/>
2. Google Generative AI (Gemini) Documentation
<https://ai.google.dev/>
3. Python Official Documentation
<https://docs.python.org/3/>
4. SQLAlchemy Documentation
<https://docs.sqlalchemy.org/>
5. Uvicorn ASGI Server Documentation
<https://www.uvicorn.org/>

6. Jinja2 Template Engine Documentation

<https://jinja.palletsprojects.com/>

7. SQLite Official Documentation

<https://www.sqlite.org/docs.html>

8. Git Version Control Documentation

<https://git-scm.com/doc>

9. Mermaid Diagram Documentation

<https://mermaid.js.org/>

10.W3Schools Web Development Tutorials

<https://www.w3schools.com/>

21. APPENDIX

- GitHub repository link : [code4nigel/fitbuddy-ai](https://github.com/code4nigel/fitbuddy-ai)
- Demo link : <https://youtu.be/YrTP66IFRa8?si=mMHvSxVrWyxEFlsr>